DATA SCIENCE & ANALYTICS · SESSION 10

ML Foundations & the sklearn Pipeline

The shared vocabulary and the reproducible workflow behind every model

Asst.Prof.Dr. Chaiwat N. · 2026

2026 · Bachelor Program · 3 hrs · Workshop Format · Hands-on Coding

Session Agenda

01 **What Machine Learning Is**
Supervised vs unsupervised; regression vs classification

02 **The ML Workflow**
Features, labels, split, fit, evaluate

03 **The sklearn API**
fit, predict, score — one pattern for everything

04 **Train/Test & Cross-Validation**
Honest evaluation, and avoiding leakage

05 **The Pipeline**
Chaining preprocessing and model, leak-free

06 **Workshop Block**
Build a full pipeline end to end — graded

01

PART 01

What Machine Learning Is

Machine learning finds patterns from data to make predictions. Before any code, get the map: the kinds of learning and the kinds of problems.

Machine learning is pattern-finding with a purpose — predict, don't just describe

The lay of the land

Supervised	Learn from labeled examples to predict a label. Most business ML — churn, revenue, fraud — is supervised.
Unsupervised	No labels; find structure. You already did this — K-Means clustering in Session 9 was unsupervised ML.
Regression	Predict a number: next month's revenue, a house price. The output is continuous.
Classification	Predict a category: churn yes/no, which segment. The output is a label.

Source: DS Course, Session 10, 2026

02

PART 02

The ML Workflow

Every supervised model follows the same five steps. Learn the workflow once and every algorithm slots into the same frame.

Every model, the same five steps — the algorithm is just one of them

Features → label → split → fit → evaluate

Features (X)	The input columns the model learns from. Good features matter more than a fancy algorithm — this is most of the craft.
Label (y)	The thing you're predicting. Supervised learning needs known answers to learn from.
Split	Hold out a test set the model never sees in training, so you can measure honest performance.
Fit & evaluate	Fit on train, predict on test, score with the right metric. Then iterate on features, not on the test set.

Source: DS Course, Session 10, 2026

03

PART 03

The sklearn API

Scikit-learn's genius is one consistent interface: every model — regression, trees, clustering — is fit, predict, score. Learn it once.

The universal pattern: fit, predict, score

One API for every model

fit

Learns from the training data. km.fit, model.fit — the same method name on every estimator in the library.

predict

Applies the learned model to new data. Swap LinearRegression for RandomForest and this line is unchanged.

score

Returns a default metric. The consistency means switching algorithms is a one-word change, not a rewrite.

PYTHON · SKLEARN API

```
from sklearn.linear_model import LinearRegression

model = LinearRegression()
model.fit(X_train, y_train)

preds = model.predict(X_test)
model.score(X_test, y_test)   # R^2

# swap the model, keep the pattern
```

fit -> predict -> score. The same on every estimator.

04

PART 04

Train/Test & Cross-Validation

A model graded on data it trained on always looks great and means nothing. Train/test split and cross-validation give you the honest number.

Split and cross-validate: measure what the model does on unseen data

Honest evaluation

train_test_split

Hold back, say, 20% as a test set. Train on the rest; the test score estimates real-world performance.

Why it matters

Score on training data and overfitting is invisible — the model memorized rather than learned.

Cross-validation

k-fold rotates the test set k times and averages. A more stable estimate than a single lucky or unlucky split.

PYTHON · SPLIT & CV

```
from sklearn.model_selection import (
    train_test_split, cross_val_score)

X_train, X_test, y_train, y_test = (
    train_test_split(X, y, test_size=0.2,
                     random_state=42))

cross_val_score(model, X, y, cv=5).mean()
```

Never trust a score measured on training data.

05

PART 05

The Pipeline

Scaling or imputing before you split leaks test information into training. A Pipeline chains every step so it's fit on train only — leak-free by construction.

The Pipeline: chain preprocessing and model into one leak-free object

Why pipelines exist

The leak

Scale using the whole dataset's mean and the test set has tainted training. Subtle, common, and fatal to honesty.

The fix

A Pipeline learns every transform on the training fold only, then applies it. Leakage becomes impossible by design.

One object

Fit it, predict with it, cross-validate it as a unit. The preprocessing travels with the model forever.

PYTHON · PIPELINE

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer

pipe = Pipeline([
    ('impute', SimpleImputer()),
    ('scale',  StandardScaler()),
    ('model',  LinearRegression()),
])
pipe.fit(X_train, y_train)
```

The Pipeline makes data leakage structurally impossible.

06

PART 06

Workshop Block

Build one sklearn Pipeline — impute, scale, model — fit it, split honestly, and evaluate. This is graded coding check #3.

By the end of this block, every student has a full, leak-free pipeline from raw data to a score

Graded Check · 20 Minutes

Assemble X and y	Choose the features and the target. Confirm shapes line up and the target is what you mean to predict.
Split	train_test_split with a fixed random_state so your run is reproducible and gradable.
Build the pipeline	Imputer → scaler → model in one Pipeline object. Fit on train only.
Evaluate	Score on the test set and cross-validate. Report both numbers. Pipeline plus scores is your deliverable.

Source: Workshop, Session 10, 2026

KEY TAKEAWAY

The workflow makes the model honest. The Pipeline keeps it that way.